

# WonderFullyHE: математическое описание API

## Назначение библиотеки

WonderFullyHE реализует библиотечный слой для защищённых вычислений над вещественными данными на базе схемы CKKS из Microsoft SEAL. Публичный API скрывает низкоуровневые типы SEAL и предоставляет:

- настройку CKKS-профилей;
- кодирование, шифрование, вычисления, расшифрование и декодирование;
- контроль численной ошибки;
- ABFT-проверки корректности вычислений;
- диагностику вычислительной глубины и bootstrapping-конвейер.

## CKKS-профиль

Профиль CKKS задаётся набором

$$P = (N, Q, \Delta, S),$$

где:

- $N$  — степень полиномиального модуля;
- $Q = (q_0, q_1, \dots, q_l)$  — цепочка коэффициентных модулей, в коде задаётся битовыми размерами;
- $\Delta$  — масштаб CKKS;
- $S$  — число используемых слотов,  $S \leq N/2$ .

В проекте используются два основных профиля:

| Профиль    | $N$   | $S$  | $Q$ , бит                   | $\Delta$ | Оценка глубины |
|------------|-------|------|-----------------------------|----------|----------------|
| basic_ckks | 8192  | 4096 | 60 – 40 – 40 – 60           | $2^{40}$ | 2              |
| depth_ckks | 16384 | 8192 | 60 – 40 – 40 – 40 – 40 – 60 | $2^{40}$ | 4              |

# Адаптер над Microsoft SEAL

Создание контекста:

$$\text{SealAdapter}::\text{create}(P) \mapsto \text{SEALContext}(P).$$

Генерация ключей:

$$\text{keygen}(r, g) \mapsto (\text{sk}, \text{pk}, \text{rlk}^{(r)}, \text{gk}^{(g)}),$$

где  $r, g \in \{0, 1\}$  задают необходимость Relin- и Galois-ключей.

Кодирование и шифрование:

$$\text{encode}(\mathbf{x}) = \text{Encode}_{\Delta}(\mathbf{x}) = p,$$

$$\text{encrypt}(p) = \text{Enc}_{\text{pk}}(p) = c.$$

Расшифрование и декодирование:

$$\text{decrypt}(c) = \text{Dec}_{\text{sk}}(c) = p,$$

$$\text{decode}(p) = \text{Decode}_{\Delta}(p) = \hat{\mathbf{x}}.$$

## Гомоморфные операции

Сложение:

$$\text{add}(c_1, c_2) = c_1 + c_2.$$

Вычитание:

$$\text{sub}(c_1, c_2) = c_1 - c_2.$$

Умножение с релинеаризацией и rescale:

$$\text{mul\_relin\_rescale}(c_1, c_2) = \text{Rescale}(\text{Relin}_{\text{rlk}}(c_1 \cdot c_2)).$$

При умножении масштаб растёт:

$$\Delta \cdot \Delta = 2^{40} \cdot 2^{40} = 2^{80}.$$

Операция **Rescale** делит результат на следующий примерно 40-битный модуль и возвращает масштаб к значению, близкому к  $2^{40}$ . При этом расходуется один уровень modulus chain.

Ротация CKKS-слотов:

$$\text{rotate}(c, k) = \text{Rot}_k(c).$$

## Диагностика ciphertext

Для ciphertext  $c$  библиотека возвращает набор параметров:

$$\text{info}(c) = (\text{scale}(c), \text{chain\_index}(c), \text{coeff\_modulus\_size}(c), \text{ciphertext\_size}(c)).$$

Также измеряется сериализованный размер:

$$\text{serialized\_size}(c) = |\text{Serialize}(c)|.$$

Эти значения используются в benchmark, анализе глубины и bootstrapping-отчёте.

## Контроль точности

Пусть  $\mathbf{y}$  — эталонный результат CPU-вычисления, а  $\hat{\mathbf{y}}$  — результат после гомоморфного вычисления, расшифрования и декодирования. Для векторов одинаковой длины  $n$ :

$$\text{max\_abs\_error} = \max_{0 \leq i < n} |y_i - \hat{y}_i|,$$

$$\text{mean\_abs\_error} = \frac{1}{n} \sum_{i=0}^{n-1} |y_i - \hat{y}_i|.$$

Критерий прохождения проверки:

$$\text{ok} = \text{max\_abs\_error} \leq \text{tolerance}.$$

## AVFT-проверки

Контрольная сумма полезной нагрузки:

$$\text{checksum}(\mathbf{x}) = \sum_{i=0}^{m-1} x_i.$$

Добавление checksum-слота:

$$\mathbf{x}' = (x_0, x_1, \dots, x_{m-1}, \text{checksum}(\mathbf{x})).$$

Проверка добавленного checksum-слота:

$$\text{expected} = \sum_{i=0}^{m-1} y_i, \quad \text{observed} = y_m,$$

$$\text{abs\_error} = |\text{expected} - \text{observed}|, \quad \text{ok} = \text{abs\_error} \leq \text{tolerance}.$$

Для сложения и вычитания используются линейные свойства:

$$\text{checksum}(\mathbf{x} + \mathbf{y}) = \text{checksum}(\mathbf{x}) + \text{checksum}(\mathbf{y}),$$

$$\text{checksum}(\mathbf{x} - \mathbf{y}) = \text{checksum}(\mathbf{x}) - \text{checksum}(\mathbf{y}).$$

Для ротации используется инвариант суммы:

$$\text{checksum}(\text{Rot}_k(\mathbf{x})) = \text{checksum}(\mathbf{x}).$$

Для умножения текущая проверка сравнивает сумму результата с CPU-эталоном покомпонентного произведения:

$$\text{expected} = \sum_{i=0}^{m-1} x_i y_i, \quad \text{observed} = \sum_{i=0}^{m-1} (\widehat{xy})_i.$$

## Анализ вычислительной глубины

В профиле `depth_ckks` библиотека анализирует цепочку последовательных зашифрованных возведений в квадрат:

$$x \rightarrow x^2 \rightarrow x^4 \rightarrow x^8 \rightarrow x^{16}.$$

Каждый шаг вызывает:

$$c_{i+1} = \text{mul\_relin\_rescale}(c_i, c_i).$$

После каждого успешного шага фиксируются:

$$\text{scale}(c_i), \quad \text{chain\_index}(c_i), \quad \text{serialized\_size}(c_i), \quad \text{error}(c_i).$$

На текущем профиле следующий переход к  $x^{32}$  останавливается, потому что доступные уровни `modulus chain` для очередного `rescale` исчерпаны.

## Bootstrapping-модуль

Bootstrapping-модуль выделен как отдельный компонент библиотеки. Он связан с анализом глубины и фиксирует конвейер:

$$\text{ModRaise} \rightarrow \text{CoeffToSlot} \rightarrow \text{EvalMod} \rightarrow \text{SlotToCoeff}.$$

Цель итогового преобразования:

$$\text{Bootstrap}(c) = c',$$

где должны выполняться условия:

$$\text{Dec}(c') \approx \text{Dec}(c),$$

$$\text{level}(c') > \text{level}(c).$$

В публичном отчёте `BootstrapReport` фиксируются:

- параметры входного `ciphertext`;
- параметры `ciphertext` на границе вычислительной глубины;
- число успешно выполненных умножений;
- следующий показатель степени, на котором вычисление останавливается;
- причина остановки;
- стадии конвейера и их статусы;
- состояние критериев  $\text{Dec}(c') \approx \text{Dec}(c)$  и  $\text{level}(c') > \text{level}(c)$ .

## Отчёт по криптостойкости профилей

Для CKKS-профиля вычисляется суммарный размер коэффициентного модуля:

$$\log_2(Q) \approx \sum_i \text{coeff\_modulus\_bits}_i.$$

Для каждого профиля библиотека сравнивает это значение с лимитами Microsoft SEAL:

$$\log_2(Q) \leq \text{CoeffModulus} :: \text{MaxBitCount}(N, \text{sec\_level}).$$

В отчёте фиксируются проверки для уровней:

$$\text{tc128}, \quad \text{tc192}, \quad \text{tc256}.$$

Для текущих профилей:

| Профиль    | $N$   | $\log_2(Q)$ | tc128 | tc192 | tc256 |
|------------|-------|-------------|-------|-------|-------|
| basic_ckks | 8192  | 200         | PASS  | FAIL  | FAIL  |
| depth_ckks | 16384 | 280         | PASS  | PASS  | FAIL  |

Общий уровень проекта определяется минимальным уровнем среди используемых профилей:

$$\min(\text{tc128}, \text{tc192}) = \text{tc128}.$$

## Демонстрационные приложения

В проекте поддерживаются следующие запускные сценарии:

| Приложение              | Назначение                                              |
|-------------------------|---------------------------------------------------------|
| demo_secure_stats       | Защищённое вычисление суммы и среднего.                 |
| demo_abft               | Проверка ABFT-инвариантов для add, sub, mul и rotate.   |
| demo_noise_growth       | Анализ расхода глубины и роста ошибки.                  |
| demo_bootstrap_pipeline | Отчёт bootstrapping-модуля.                             |
| bench_ckks              | Измерение времени, ошибки и размеров ciphertext/ключей. |
| demo_profile_report     | Таблица используемых CKKS-профилей.                     |
| demo_security_report    | Проверка профилей по лимитам Microsoft SEAL.            |